OAVS TGT COMPUTER SCIENCE TEACHER EXAMINATION

MCQ Question Bank

Topic: Storage Classes in C

148 Unique, Exam-Oriented Multiple Choice Questions

auto • register • static (local & global) • extern • scope rules

25	25	25	20
AUTOMATIC STOR	REGISTER STORA	STATIC STORAGE	STATIC STORAGE

Every question includes the correct answer and a revision-focused explanation

Table of Contents

1. Automatic Storage Class (auto)	25 questions
2. Register Storage Class (register)	25 questions
3. Static Storage Class — Local Variables	25 questions
4. Static Storage Class — Global Variables	20 questions
5. External Storage Class (extern)	25 questions
6. Scope Rules & Storage Class Comparison	28 questions
Total Questions	**148**

1. Automatic Storage Class (auto) 25 Qs

Q1 | Definition | Automatic Storage Class (auto)

Which is the default storage class for local variables declared inside a function/block in C?

A. extern
B. static
C. register
D. auto

Ans: D. auto

Explanation: If no storage class is specified for a local variable, the C compiler treats it as **auto** by default.

Q2 | Conceptual | Automatic Storage Class (auto)

In which memory area are automatic (auto) variables stored?

A. Stack
B. CPU registers
C. Data segment
D. Heap segment

Ans: A. Stack

Explanation: Auto variables are created on the function call stack and are removed automatically when the block/function exits.

Q3 | Lifetime | Automatic Storage Class (auto)

What is the lifetime of an auto variable?

A. Until the program restarts
B. Entire program execution
C. Until the block/function in which it is declared finishes execution
D. Permanent, across all function calls

Ans: C. Until the block/function in which it is declared finishes execution

Explanation: An auto variable is created when its block is entered and destroyed as soon as control leaves that block.

Q4 | Scope | Automatic Storage Class (auto)

What is the scope of an automatic variable?

A. File scope
B. Global to the whole program
C. Scope of the entire project
D. Local to the block/function in which it is declared

Ans: D. Local to the block/function in which it is declared

Explanation: Auto variables are visible only within the block in which they are declared; they cannot be accessed outside it.

Q5 | Declaration | Automatic Storage Class (auto)

Which of the following is a syntactically correct declaration of an auto variable?

A. int auto x;
B. x auto int;
C. auto x int;
D. auto int x;

Ans: D. auto int x;

Explanation: The storage class keyword is written before the data type: *storage_class data_type variable_name;*

Q6 | Characteristics | Automatic Storage Class (auto)

An automatic variable that is declared but not explicitly initialized contains:

A. The value -1
B. The value 0
C. NULL
D. An unpredictable garbage value

Ans: D. An unpredictable garbage value

Explanation: Unlike static/global variables, auto variables are not automatically initialized to zero; they hold garbage values until assigned.

Q7 | Application | Automatic Storage Class (auto)

The auto storage class is best suited for:

A. Variables meant to be stored in CPU registers
B. Variables shared across multiple files
C. Variables that must retain their value across function calls
D. Temporary variables required only within a function

Ans: D. Temporary variables required only within a function

Explanation: Since auto variables exist only for the duration of the block, they are ideal for short-lived, temporary working variables.

Q8 | Statement-based | Automatic Storage Class (auto)

Statement: "auto variables are created when the block is entered and destroyed when the block exits." This statement is:

A. True
B. True only for global auto variables
C. True only in main()
D. False

Ans: A. True

Explanation: Each call to the block allocates new stack memory for auto variables, so previous values are not retained.

Q9 | Conceptual | Automatic Storage Class (auto)

Can the keyword 'auto' be used to declare a variable at global (file) scope?

A. Yes, it behaves like a global variable
B. Yes, but only in the first file
C. Yes, if extern is also used
D. No, auto is invalid for variables declared outside all functions

Ans: D. No, auto is invalid for variables declared outside all functions

Explanation: The auto storage class applies only to local variables; using it at file scope results in a compile-time error.

Q10 | Tricky | Automatic Storage Class (auto)

What happens if you explicitly write 'auto' before a variable declared outside any function?

A. It is treated as extern
B. It is silently accepted and treated as global
C. It is treated as static
D. It causes a compile-time error

Ans: D. It causes a compile-time error

Explanation: auto is meaningless outside a block/function, and most compilers report an error for such a declaration.

Q11 | Memory allocation | Automatic Storage Class (auto)

Each time a function containing an auto variable is called:

A. Memory is allocated in the data segment

B. Memory is allocated from the heap

C. Fresh stack memory is allocated for the variable

D. The variable reuses the same memory address as the previous call

Ans: C. Fresh stack memory is allocated for the variable

Explanation: Stack memory for auto variables is allocated on entry to the function and released on exit, so a new call gets fresh memory.

Q12 | Code-based | Automatic Storage Class (auto)

What will the following code print if func() is called twice?

```
void func(){ auto int x = 5; x++; printf("%d ", x); }
```

A. 5 6

B. 5 5

C. 6 6

D. 6 7

Ans: C. 6 6

Explanation: Since x is reinitialized to 5 on every call (auto variables don't persist), each call prints 6.

Q13 | Output-based | Automatic Storage Class (auto)

Predict the output:

```
int i; for(i=0;i<3;i++){ auto int c=0; c++; printf("%d",c);}
```

A. 123

B. 000

C. 111

D. 012

Ans: C. 111

Explanation: 'c' is declared and initialized to 0 afresh in every iteration of the loop since it is auto, so it becomes 1 each time before printing.

Q14 | Assertion-Reason | Automatic Storage Class (auto)

Assertion (A): Auto variables lose their value once the function returns.
Reason (R): Auto variables are stored in stack memory, which is deallocated when the function exits.

A. A is true, R is false

B. Both A and R are true, and R is the correct explanation of A

C. Both A and R are true, but R is not the correct explanation of A

D. A is false, R is true

Ans: B. Both A and R are true, and R is the correct explanation of A

Explanation: The stack frame (and hence the auto variable's memory) is destroyed on function return, which directly explains why its value is lost.

Q15 | Fill-in-the-blank | Automatic Storage Class (auto)

The storage class keyword _____ is rarely typed explicitly in programs because it is the implicit default for local variables.

A. register
B. auto
C. static
D. extern

Ans: B. auto

Explanation: Since 'auto' is applied automatically to local variables, programmers usually omit it.

Q16 | Comparison | Automatic Storage Class (auto)

Which of the following is NOT a characteristic of the auto storage class?

A. Value persists between successive function calls
B. Stored in stack memory
C. Default storage class for local variables
D. Block/local scope

Ans: A. Value persists between successive function calls

Explanation: Auto variables do NOT retain their value between calls — that persistence is a property of static variables, not auto.

Q17 | Scenario-based | Automatic Storage Class (auto)

A programmer declares 'int count;' inside a for-loop body without any storage class keyword. What storage class does 'count' get by default?

A. register
B. extern
C. auto
D. static

Ans: C. auto

Explanation: Any local variable without an explicit storage-class specifier is automatic (auto) by default.

Q18 | Previous-Year Pattern | Automatic Storage Class (auto)

The storage class that provides block scope, stack-based memory, and automatic destruction at the end of the block is called:

A. static
B. auto
C. register
D. extern

Ans: B. auto

Explanation: This combination of properties — block scope, stack memory, automatic destruction — precisely defines the auto storage class.

Q19 | Difficult | Automatic Storage Class (auto)

Which of the following statements about auto variables is FALSE?

A. The auto keyword can be used explicitly with local variables

B. Auto variables are destroyed when control leaves their block

C. Explicitly writing 'auto' is mandatory for every local variable

D. Auto variables have block/local scope

Ans: C. Explicitly writing 'auto' is mandatory for every local variable

Explanation: Writing 'auto' explicitly is optional, not mandatory, since it is the default for local declarations.

Q20 | Definition | Automatic Storage Class (auto)

An automatic variable is also sometimes referred to as a:

A. Global variable

B. Register variable

C. Local/temporary variable

D. Persistent variable

Ans: C. Local/temporary variable

Explanation: Because of its limited lifetime and scope, an auto variable is commonly called a local or temporary variable.

Q21 | Conceptual | Automatic Storage Class (auto)

If two different functions each declare an auto variable named 'x', are these two variables related?

A. Yes, they share the same memory

B. Yes, but only if both functions are called from main()

C. No, they are completely independent variables with separate memory

D. No, only one of them is valid

Ans: C. No, they are completely independent variables with separate memory

Explanation: Each function has its own local scope; auto variables with the same name in different functions occupy separate stack memory.

Q22 | Statement-based | Automatic Storage Class (auto)

Statement: "Auto variables can be accessed from a function other than the one in which they are declared." This is:

A. True

B. False

C. True only for pointers

D. True only if declared in main()

Ans: B. False

Explanation: Auto variables have local scope and are inaccessible outside their declaring block/function.

Q23 | Application | Automatic Storage Class (auto)

Which of these is a typical, appropriate use of an auto variable?

A. A configuration flag needed by every function in the file

B. A variable that must be optimized into a CPU register at all costs

C. A variable meant to be shared between multiple source files

D. A loop counter used only within a single function

Ans: D. A loop counter used only within a single function

Explanation: Loop counters that are only needed within one function are classic, appropriate use cases for auto variables.

Q24 | Tricky | Automatic Storage Class (auto)

What is the initial value of an uninitialized auto variable of type int?

A. Indeterminate (garbage) value

B. Always -1

C. Always the value of the last variable in that memory location, guaranteed by the standard

D. Always 0

Ans: A. Indeterminate (garbage) value

Explanation: The C standard does not guarantee any specific value for an uninitialized auto variable; it is indeterminate/ garbage.

Q25 | Comparison | Automatic Storage Class (auto)

Auto variables differ from static local variables mainly in terms of:

A. The keyword used for declaration only, with no functional difference

B. Only in the syntax of printf used with them

C. Data type restrictions

D. Lifetime and value persistence across function calls

Ans: D. Lifetime and value persistence across function calls

Explanation: The key functional difference is that auto variables are destroyed after each call, while static local variables persist their value across calls.

2. Register Storage Class (register) 25 Qs

Q26 | Definition | Register Storage Class (register)

The 'register' storage class is a request to the compiler to store a variable in:

A. A file on disk
B. The heap
C. The data segment
D. A CPU register, for faster access

Ans: D. A CPU register, for faster access

Explanation: The register keyword suggests that the compiler place the variable in a CPU register for quicker access, though the compiler may not honor this.

Q27 | Conceptual | Register Storage Class (register)

Is the compiler obligated to store a 'register' variable in an actual CPU register?

A. Only for float variables
B. Yes, always
C. Only inside main()
D. No, the compiler may ignore the request and treat it as auto if registers are unavailable

Ans: D. No, the compiler may ignore the request and treat it as auto if registers are unavailable

Explanation: The register specifier is only a hint/suggestion to the compiler; if no register is free, the variable is treated like an ordinary auto variable.

Q28 | Declaration | Register Storage Class (register)

Which of the following is a valid declaration using the register storage class?

A. int register counter;
B. register int counter;
C. counter register int;
D. Both (a) and (b) are valid

Ans: D. Both (a) and (b) are valid

Explanation: The storage-class specifier may appear either before or after the type specifier, so both forms are valid C syntax.

Q29 | Advantage | Register Storage Class (register)

What is the main advantage claimed for register variables?

A. They have global scope
B. They occupy less memory than int variables
C. Faster access/execution since CPU registers are quicker than RAM
D. They are automatically initialized to zero

Ans: C. Faster access/execution since CPU registers are quicker than RAM

Explanation: Accessing a value directly from a CPU register avoids a memory (RAM) access, which can speed up execution for frequently used variables.

Q30 | Limitation | Register Storage Class (register)

Which operation is NOT allowed on a variable declared with the register storage class?

A. Using it in a comparison
B. Arithmetic operations like addition
C. Assigning it a value
D. Taking its address using the & operator

Ans: D. Taking its address using the & operator

Explanation: Because a register variable may not have a normal memory address, the unary '&' (address-of) operator cannot be applied to it.

Q31 | Scope | Register Storage Class (register)

What is the scope of a register variable?

A. Restricted only to CPU hardware routines
B. Program-wide
C. Local to the block in which it is declared, same as auto
D. Global to the file

Ans: C. Local to the block in which it is declared, same as auto

Explanation: Like auto variables, register variables are local to the block in which they are declared.

Q32 | Lifetime | Register Storage Class (register)

What is the lifetime of a register variable?

A. Same as an extern variable
B. Permanent, stored for the OS lifetime
C. Same as a static variable — throughout the program
D. Same as an auto variable — until the block/function ends

Ans: D. Same as an auto variable — until the block/function ends

Explanation: Register variables behave like auto variables in terms of lifetime, existing only for the duration of their enclosing block.

Q33 | Application | Register Storage Class (register)

Register storage class is typically applied to variables that are:

A. Meant to be shared across files
B. Large arrays and structures
C. Rarely used in the program
D. Frequently accessed, such as loop counters

Ans: D. Frequently accessed, such as loop counters

Explanation: Frequently accessed variables, like loop counters in performance-critical loops, are the classic candidates for the register hint.

Q34 | Limitation | Register Storage Class (register)

Why can large data types like arrays or structures generally NOT be effectively declared as register variables?

A. Because arrays cannot be declared with any storage class
B. Because CPU registers are typically too small to hold them
C. Because register only applies to global variables
D. Because the compiler forbids syntax for arrays with register

Ans: B. Because CPU registers are typically too small to hold them

Explanation: CPU registers have limited size, so bulky data such as arrays/structures usually cannot fit into a register and the request is ignored.

Q35 | Conceptual | Register Storage Class (register)

In modern C compilers, the register keyword is often considered:

A. Illegal and removed from the language

B. Largely obsolete, since compilers automatically optimize variable placement

C. Only usable with pointers

D. Mandatory for performance

Ans: B. Largely obsolete, since compilers automatically optimize variable placement

Explanation: Modern optimizing compilers generally decide register allocation on their own, making the explicit register keyword mostly a legacy hint.

Q36 | Statement-based | Register Storage Class (register)

Statement: "A register variable is guaranteed to always be faster than an auto variable." This statement is:

A. True in all cases

B. True only in loops

C. False, since the compiler may not place it in a register at all

D. True only for char variables

Ans: C. False, since the compiler may not place it in a register at all

Explanation: Since the compiler can silently ignore the register request, there is no guarantee of any speed benefit.

Q37 | Tricky | Register Storage Class (register)

What is the result of attempting to compile: 'register int x = 10; int *p = &x;'?

A. Compiles with a warning but assigns garbage to p

B. Runtime crash only

C. Compile-time error because the address of a register variable cannot be taken

D. Compiles and runs normally

Ans: C. Compile-time error because the address of a register variable cannot be taken

Explanation: The C standard disallows applying the address-of operator to a variable declared as register.

Q38 | Code-based | Register Storage Class (register)

Which line in the following code is invalid?

```
register int a = 5;
int *ptr = &a;
```

A. Neither line is invalid

B. register int a = 5;

C. int *ptr = &a;

D. Both lines are invalid

Ans: C. int *ptr = &a;

Explanation: Taking the address of register variable 'a' is not permitted, making the second line invalid.

Q39 | Comparison | Register Storage Class (register)

Which pair correctly matches storage class with primary storage location suggested?

A. register - CPU register

B. static - CPU register

C. extern - CPU register

D. auto - CPU register

Ans: A. register - CPU register

Explanation: The register storage class specifically suggests CPU register storage, unlike auto, static, or extern.

Q40 | Assertion-Reason | Register Storage Class (register)

Assertion (A): register variables cannot have their address taken.
Reason (R): CPU registers do not have conventional memory addresses like RAM locations.

A. Both A and R are true, but R does not explain A
B. Both A and R are true, and R correctly explains A
C. A is true, R is false
D. A is false, R is true

Ans: B. Both A and R are true, and R correctly explains A

Explanation: Since registers lack ordinary addressable memory locations, the & operator cannot be meaningfully applied, explaining the restriction.

Q41 | Fill-in-the-blank | Register Storage Class (register)

The register storage class is only a _____ to the compiler, not a command.

A. command
B. restriction
C. suggestion/request
D. guarantee

Ans: C. suggestion/request

Explanation: The keyword register merely requests register allocation; the compiler is free to disregard it.

Q42 | Difficult | Register Storage Class (register)

If a program declares more register variables than the number of available CPU registers, what happens?

A. Compilation fails
B. The compiler treats the excess variables as ordinary auto variables
C. Only the first register variable is retained; others are deleted
D. The program crashes at runtime

Ans: B. The compiler treats the excess variables as ordinary auto variables

Explanation: When registers run out, the compiler gracefully falls back to treating additional register variables as auto.

Q43 | Scenario-based | Register Storage Class (register)

A programmer wants to speed up a loop counter that runs a million iterations. Which storage class hint might they apply, keeping in mind it may be ignored by the compiler?

A. static
B. register
C. extern
D. auto is the only valid choice

Ans: B. register

Explanation: register is the traditional hint used for performance-critical, frequently accessed variables like loop counters.

Q44 | Conceptual | Register Storage Class (register)

Which of these is true regarding register variables and function scope?

A. They exist for the life of the program
B. They are local, exactly like auto variables, restricted to their own block
C. They can be accessed from any function in the file
D. They must be declared only in main()

Ans: B. They are local, exactly like auto variables, restricted to their own block

Explanation: Apart from the register hint, scope rules for register variables mirror those of auto variables.

Q45 | Previous-Year Pattern | Register Storage Class (register)

Which of the following statements is TRUE about the register storage class in C?

A. It automatically retains value between function calls

B. It has the same scope and lifetime as an auto variable, but a different storage suggestion

C. It can be used only for global variables

D. It guarantees storage in a CPU register

Ans: B. It has the same scope and lifetime as an auto variable, but a different storage suggestion

Explanation: register shares auto's scope and lifetime rules; the only difference is the (non-binding) suggestion to use a CPU register.

Q46 | Tricky | Register Storage Class (register)

Why might using 'register' on too many variables sometimes have no visible effect on program performance?

A. Because register variables are always slower than auto

B. Because register variables cannot be used in arithmetic expressions

C. Because using register disables all compiler optimizations

D. Because modern compilers already apply their own optimizations and may ignore explicit register requests

Ans: D. Because modern compilers already apply their own optimizations and may ignore explicit register requests

Explanation: Contemporary compilers use sophisticated register-allocation algorithms, often outperforming or overriding manual register hints.

Q47 | Comparison | Register Storage Class (register)

Which is a key difference between register and auto variables?

A. register variables are static, auto variables are not

B. There is no difference at all

C. register variables cannot have their address taken, unlike auto variables

D. register variables have global scope while auto variables do not

Ans: C. register variables cannot have their address taken, unlike auto variables

Explanation: Address-of (&) is disallowed on register variables — a restriction that does not apply to ordinary auto variables.

Q48 | Definition | Register Storage Class (register)

The register storage class was introduced in C primarily to:

A. Improve execution speed by hinting frequently used variables for CPU register storage

B. Allow variables to be shared across files

C. Enable static memory allocation

D. Reduce program size

Ans: A. Improve execution speed by hinting frequently used variables for CPU register storage

Explanation: Its original design goal was to give the programmer a way to hint performance-critical variables to the compiler.

Q49 | Statement-based | Register Storage Class (register)

Statement: "All register variables must be declared inside a function or block." This statement is:

A. False
B. True only for pointer variables
C. True
D. True only in single-file programs

Ans: C. True

Explanation: Just like auto, the register storage class is applicable only to local variables inside functions/blocks, not at file scope.

Q50 | Application | Register Storage Class (register)

In embedded or performance-critical C code, the register keyword is historically applied to:

A. Small, frequently accessed variables like counters or indices
B. Externally linked variables
C. File-scope global constants
D. Large data structures

Ans: A. Small, frequently accessed variables like counters or indices

Explanation: Small, heavily used variables such as loop indices are the ideal (traditional) candidates for the register hint.

3. Static Storage Class — Local Variables

25 Qs

Q51 | Definition | Static Storage Class — Local Variables

Which storage class allows a local variable to retain its value between successive calls to the function?

A. static
B. extern
C. register
D. auto

Ans: A. static

Explanation: A static local variable preserves its value across function calls because its memory is allocated only once, for the program's lifetime.

Q52 | Concept | Static Storage Class — Local Variables

A static local variable is initialized:

A. Only once, the first time the function is called
B. Twice — once at compile time and once at runtime
C. Every time the function is called
D. Never, it must be assigned manually every time

Ans: A. Only once, the first time the function is called

Explanation: Static local variables are initialized only once, on the first call; subsequent calls skip the initialization statement.

Q53 | Declaration | Static Storage Class — Local Variables

Which is the correct syntax to declare a static local integer variable initialized to 0?

A. x static int = 0;
B. Both (a) and (b) are valid
C. int static x = 0;
D. static int x = 0;

Ans: B. Both (a) and (b) are valid

Explanation: The storage-class specifier may precede or follow the type specifier, so both forms are syntactically valid.

Q54 | Lifetime | Static Storage Class — Local Variables

What is the lifetime of a static local variable?

A. Only during compilation
B. Throughout the entire execution of the program
C. Only during the function call in which it is declared
D. Until another function with the same variable name is called

Ans: B. Throughout the entire execution of the program

Explanation: Once created, a static local variable exists for the whole program run, even though it is only accessible inside its function.

Q55 | Scope | Static Storage Class — Local Variables

What is the scope of a static local variable?

A. Local to the function/block in which it is declared
B. Accessible from any file in the project
C. File scope across the whole program
D. Global to the entire program

Ans: A. Local to the function/block in which it is declared

Explanation: Despite its program-long lifetime, a static local variable's scope remains restricted to its declaring function/block.

Q56 | Memory behavior | Static Storage Class — Local Variables

In which memory segment is a static local variable stored?

A. Heap segment
B. Data segment (static/global memory area)
C. CPU register
D. Stack segment

Ans: B. Data segment (static/global memory area)

Explanation: Static variables (local or global) are allocated in the data segment, not the stack, which is why they persist across calls.

Q57 | Characteristics | Static Storage Class — Local Variables

If not explicitly initialized, a static local variable of type int is automatically set to:

A. 0
B. 1
C. -1
D. Garbage value

Ans: A. 0

Explanation: Unlike auto variables, static variables are automatically zero-initialized by the compiler if no explicit initializer is given.

Q58 | Code-based | Static Storage Class — Local Variables

What is the output of the following code when func() is called three times?

```
void func(){ static int count = 0; count++; printf("%d ", count); }
```

A. 1 2 3
B. 1 1 1
C. 3 3 3
D. 0 1 2

Ans: A. 1 2 3

Explanation: Because count is static, it retains its value between calls, incrementing from 1 to 2 to 3 across the three calls.

Q59 | Output-based | Static Storage Class — Local Variables

Predict the output:

```
void show(){ static int n = 5; n += 2; printf("%d ", n); }
int main(){ show(); show(); return 0; }
```

A. 5 7
B. 7 9
C. 7 7
D. 5 5

Ans: B. 7 9

Explanation: n starts at 5 (only once), becomes 7 after the first call, and retains that value, becoming 9 after the second call.

Q60 | Application | Static Storage Class — Local Variables

Static local variables are commonly used to:

A. Replace all global variables in a program

B. Count how many times a function has been called across the program

C. Store CPU-register-only values

D. Store temporary loop indices

Ans: B. Count how many times a function has been called across the program

Explanation: A classic use of a static local variable is a call-counter that persists its count between invocations of the function.

Q61 | Statement-based | Static Storage Class — Local Variables

Statement: "A static local variable is re-initialized every time its declaration statement is encountered during execution." This is:

A. True only for float variables

B. False — initialization happens only once

C. True

D. True only for arrays

Ans: B. False — initialization happens only once

Explanation: Even though the declaration line executes on every call, the actual initialization is performed only the very first time.

Q62 | Assertion-Reason | Static Storage Class — Local Variables

Assertion (A): A static local variable is not destroyed when the function returns.
Reason (R): Static variables are stored in the data segment, which exists for the entire program duration.

A. Both A and R true, R is not the correct explanation

B. Both A and R true, R is the correct explanation

C. A false, R true

D. A true, R false

Ans: B. Both A and R true, R is the correct explanation

Explanation: The persistence of static variables is a direct consequence of their allocation in the permanent data segment, correctly explaining assertion A.

Q63 | Tricky | Static Storage Class — Local Variables

Can two different functions each have a static local variable with the same name without conflict?

A. No, this causes a compile-time error

B. No, only one static variable per program is allowed

C. Yes, but they will share the same memory location

D. Yes, each function's static local variable is independent, with its own storage

Ans: D. Yes, each function's static local variable is independent, with its own storage

Explanation: Static local variables are scoped to their own function, so identical names in different functions do not clash and use separate storage.

Q64 | Comparison | Static Storage Class — Local Variables

What is the key difference between an auto local variable and a static local variable?

A. Static variables have global scope; auto do not

B. There is no real difference

C. Auto variables are zero-initialized by default, static are not

D. Static variables persist their value between calls; auto variables do not

Ans: D. Static variables persist their value between calls; auto variables do not

Explanation: Persistence of value across function calls is the defining distinction between static and auto local variables.

Q65 | Fill-in-the-blank | Static Storage Class — Local Variables

A static local variable retains its previous value because it is allocated in the _____ segment rather than the stack.

A. register

B. static/data

C. code (text)

D. heap

Ans: B. static/data

Explanation: Static and global variables reside in the data segment, which is not deallocated when a function returns.

Q66 | Scenario-based | Static Storage Class — Local Variables

A programmer wants a function to remember how many times it has been invoked during the whole program run, without using a global variable. Which storage class should the counter use?

A. extern

B. auto

C. static

D. register

Ans: C. static

Explanation: A static local variable is the standard technique to keep a call-count without exposing it as a global variable.

Q67 | Previous-Year Pattern | Static Storage Class — Local Variables

Which of the following best explains why a static local variable is only initialized once?

A. Because static local variables cannot be reassigned

B. Because static variables use the register storage location

C. Because the compiler ignores subsequent initializer expressions after the first program run

D. Because static variables are allocated and initialized at program load time, not each time the block executes

Ans: D. Because static variables are allocated and initialized at program load time, not each time the block executes

Explanation: The static variable's storage and initial value are set up once when the program loads, not on every function call.

Q68 | Difficult | Static Storage Class — Local Variables

What will happen if a static local variable is declared with an initializer that calls another function, e.g. 'static int x = compute();' in standard C?

A. This is always allowed, exactly like for auto variables

B. It compiles and calls compute() on every function invocation

C. In standard C, static local variable initializers must be constant expressions, so a function call is not allowed

D. It compiles and calls compute() only at the end of the program

Ans: C. In standard C, static local variable initializers must be constant expressions, so a function call is not allowed

Explanation: Unlike auto variables, static variable initializers in C must be constant expressions evaluated at compile time, not function calls.

Q69 | Conceptual | Static Storage Class — Local Variables

If a static local variable is never explicitly reset inside the function body, its value across multiple calls will:

A. Continue changing/accumulating based on operations performed in previous calls

B. Always remain at zero

C. Become undefined after the second call

D. Reset to its initializer value every time

Ans: A. Continue changing/accumulating based on operations performed in previous calls

Explanation: Because it is not re-initialized, any modifications made in earlier calls carry forward into later calls.

Q70 | Statement-based | Static Storage Class — Local Variables

Statement: "Static local variables can be accessed directly from other functions in the same file." This is:

A. True

B. True only if declared in main()

C. False — scope remains local to the declaring function

D. True if the file has only one function

Ans: C. False — scope remains local to the declaring function

Explanation: Even though its lifetime is program-long, a static local variable's visibility (scope) is still confined to its own function.

Q71 | Tricky | Static Storage Class — Local Variables

Which of these correctly differentiates 'lifetime' from 'scope' for a static local variable?

A. Lifetime is program-long, but scope remains restricted to the declaring function/block

B. Scope is program-long, but lifetime is restricted to the function

C. Lifetime and scope are always identical for static variables

D. Both lifetime and scope are restricted to the function

Ans: A. Lifetime is program-long, but scope remains restricted to the declaring function/block

Explanation: This is the classic distinguishing feature of static local variables: extended lifetime combined with restricted (local) scope.

Q72 | Application | Static Storage Class — Local Variables

A function-local static array used to cache results between calls is a common technique known informally as:

A. Static memoization within a function
B. Register caching
C. External linkage
D. Global caching

Ans: A. Static memoization within a function

Explanation: Using a static local variable/array to remember computed results between calls is a simple form of memoization.

Q73 | Code-based | Static Storage Class — Local Variables

What is printed by:

```
void f(){ static int a; printf("%d ", a); a=10; }
int main(){ f(); f(); }
```

A. 0 10
B. 0 0
C. 10 0
D. 10 10

Ans: A. 0 10

Explanation: a is zero-initialized by default on the first call (prints 0), then set to 10; on the second call it retains 10 from before printing.

Q74 | Comparison | Static Storage Class — Local Variables

Which of the following is TRUE when comparing static local and register variables?

A. Register persists across calls; static does not
B. Both persist their value across function calls
C. Neither persists across calls
D. Static persists across calls; register does not (its lifetime matches auto)

Ans: D. Static persists across calls; register does not (its lifetime matches auto)

Explanation: Only static variables (local or global) persist values across calls; register variables behave like auto in terms of lifetime.

Q75 | Difficult | Static Storage Class — Local Variables

Why must a static local array's size be a constant expression known at compile time?

A. Because static arrays can only hold single elements
B. Because static arrays are not supported by the C language at all
C. Because static variables must have a size and layout fixed before execution begins, since they are allocated in the data segment before the program runs
D. Because arrays cannot use the static keyword

Ans: C. Because static variables must have a size and layout fixed before execution begins, since they are allocated in the data segment before the program runs

Explanation: Static storage is set up at compile/load time, so its size must be a constant expression known before the program runs.

4. Static Storage Class — Global Variables

20 Qs

Q76 | Definition | Static Storage Class — Global Variables

What is the effect of declaring a global variable as 'static' in C?

A. It is destroyed at the end of main()

B. Its scope is restricted to the file in which it is declared (internal linkage)

C. It becomes accessible from other files as well

D. It behaves exactly like a register variable

Ans: B. Its scope is restricted to the file in which it is declared (internal linkage)

Explanation: A static global variable has internal linkage, meaning it can be used only within the source file where it is defined.

Q77 | Declaration | Static Storage Class — Global Variables

Which is the correct way to declare a static global variable at file scope?

A. extern static int total;

B. int total; static;

C. register static int total;

D. static int total; // written outside all functions

Ans: D. static int total; // written outside all functions

Explanation: The static keyword is simply placed before the type in a file-scope declaration, outside any function.

Q78 | Scope | Static Storage Class — Global Variables

The scope of a static global variable is best described as:

A. Function scope only

B. Block scope

C. File scope — visible throughout the file it's declared in, but not other files

D. Program-wide across all files, like extern

Ans: C. File scope — visible throughout the file it's declared in, but not other files

Explanation: Static global variables have file scope: usable anywhere in that file, but hidden (not linked) from other files.

Q79 | Lifetime | Static Storage Class — Global Variables

What is the lifetime of a static global variable?

A. Only until the file finishes compiling

B. Same as auto — until the enclosing function returns

C. Entire duration of the program

D. Same as register variables

Ans: C. Entire duration of the program

Explanation: Like all static variables, a static global variable is created once and exists for the whole program run.

Q80 | Accessibility | Static Storage Class — Global Variables

Can a static global variable declared in file1.c be accessed in file2.c using an extern declaration?

A. Yes, always

B. Yes, if both files are compiled together into one executable, regardless of static

C. Yes, but only if extern int x; is used with #include

D. No — static gives it internal linkage, preventing access from other files

Ans: D. No — static gives it internal linkage, preventing access from other files

Explanation: The whole purpose of static at global scope is to block external linkage, so other files cannot access it even with extern.

Q81 | Characteristics | Static Storage Class — Global Variables

The main purpose of declaring a global variable as static is to:

A. Allow it to be modified only inside main()

B. Speed up program execution using CPU registers

C. Make the variable accept only integer values

D. Achieve data hiding/encapsulation by restricting visibility to a single file

Ans: D. Achieve data hiding/encapsulation by restricting visibility to a single file

Explanation: Static globals implement a basic form of encapsulation, preventing accidental name clashes or misuse from other files.

Q82 | Statement-based | Static Storage Class — Global Variables

Statement: "A static global variable and a non-static (plain) global variable both have file-wide scope within their own file." This is:

A. False

B. True only for integers

C. True only if extern is also used

D. True

Ans: D. True

Explanation: Within the same file, both static and non-static globals are visible throughout; the difference is only in cross-file linkage.

Q83 | Comparison | Static Storage Class — Global Variables

What distinguishes a static global variable from a plain (non-static) global variable?

A. There is no difference between them

B. Plain globals cannot be initialized; static globals can

C. Static globals have internal linkage (file-only); plain globals have external linkage (accessible from other files via extern)

D. Static globals are stored in the stack, plain globals in the heap

Ans: C. Static globals have internal linkage (file-only); plain globals have external linkage (accessible from other files via extern)

Explanation: Linkage is the key distinction — static restricts visibility to the file, while plain globals can be shared across files.

Q84 | Application | Static Storage Class — Global Variables

Static global variables are typically used when a programmer wants:

A. A variable with block-level scope

B. A variable to be accessible everywhere across the whole project

C. A file-private variable/helper accessible to all functions in that file but hidden from other files

D. A variable stored purely in a CPU register

Ans: C. A file-private variable/helper accessible to all functions in that file but hidden from other files

Explanation: This file-level encapsulation is the standard motivation for using static at global scope, e.g., for internal helper state.

Q85 | Conceptual | Static Storage Class — Global Variables

If two files each declare 'static int count;' independently, do they refer to the same memory location?

A. Yes, they always share memory

B. Yes, but only when linked together with extern

C. No — each file's static global variable is completely independent due to internal linkage

D. No, only one of the declarations is valid

Ans: C. No — each file's static global variable is completely independent due to internal linkage

Explanation: Internal linkage means the two static globals, though same-named, are entirely separate variables in separate files.

Q86 | Assertion-Reason | Static Storage Class — Global Variables

Assertion (A): A static global variable cannot be accessed from another source file even by declaring it extern there.
Reason (R): The static keyword gives the variable internal linkage, restricting it strictly to the translation unit (file) in which it is defined.

A. Both A and R true, R correctly explains A

B. Both A and R true, but R does not explain A

C. A false, R true

D. A true, R false

Ans: A. Both A and R true, R correctly explains A

Explanation: Internal linkage is precisely the mechanism that produces the inaccessibility described in the assertion.

Q87 | Memory behavior | Static Storage Class — Global Variables

In which memory segment is a static global variable allocated?

A. Heap

B. CPU register

C. Data/BSS segment, same as static local variables

D. Stack

Ans: C. Data/BSS segment, same as static local variables

Explanation: All static-duration variables — global or local — reside in the program's data (initialized) or BSS (uninitialized) segment.

Q88 | Fill-in-the-blank | Static Storage Class — Global Variables

A global variable with the _____ keyword is restricted in visibility to the file in which it appears, unlike an ordinary global variable.

A. extern
B. auto
C. static
D. register

Ans: C. static

Explanation: static is the keyword that limits a global variable's linkage to its own file.

Q89 | Scenario-based | Static Storage Class — Global Variables

A large project has multiple .c files, each needing a private helper variable named 'buffer' without naming conflicts across files. What should each file do?

A. Declare 'buffer' using register in each file
B. Declare 'buffer' only once in one file and use it in all files without any keyword
C. Declare 'buffer' using extern in each file
D. Declare 'buffer' as static in each file so each copy is independent and file-private

Ans: D. Declare 'buffer' as static in each file so each copy is independent and file-private

Explanation: Declaring the variable static in each file avoids linkage conflicts and keeps each file's copy private.

Q90 | Previous-Year Pattern | Static Storage Class — Global Variables

Which storage class is appropriate for a global variable meant to persist across the program but hidden from other files for safe/modular programming?

A. static
B. auto
C. register
D. None, this is not possible in C

Ans: A. static

Explanation: static at file scope offers both program-long persistence and restricted, file-only visibility.

Q91 | Difficult | Static Storage Class — Global Variables

If a static global variable is not explicitly initialized, its default value is:

A. NULL only for pointers, garbage for other types
B. 0 (zero-initialized, like all static-duration variables)
C. 1
D. Garbage/unpredictable

Ans: B. 0 (zero-initialized, like all static-duration variables)

Explanation: As with static local variables, static globals are automatically zero-initialized if no explicit value is given.

Q92 | Tricky | Static Storage Class — Global Variables

Why might a programmer prefer static global variables over plain global variables in modular, multi-file software development?

A. Static globals allow dynamic resizing of arrays

B. Static globals prevent accidental naming collisions and unintended modification from other files

C. Plain global variables are illegal in ISO C

D. Static globals execute faster in CPU registers

Ans: B. Static globals prevent accidental naming collisions and unintended modification from other files

Explanation: Restricting visibility to one file reduces the risk of naming clashes and unintended external modification — the core benefit of static globals.

Q93 | Conceptual | Static Storage Class — Global Variables

Which statement is TRUE about function calls to a static global variable from within its own file?

A. Only main() can access it

B. Only functions declared static can access it

C. Any function in that file can read/modify it directly

D. No function can access it directly, it needs a getter function

Ans: C. Any function in that file can read/modify it directly

Explanation: Within its own file, a static global variable behaves like an ordinary global — accessible to every function in that file.

Q94 | Comparison | Static Storage Class — Global Variables

Which of these correctly compares static local and static global variables?

A. There is no difference between them

B. Static local variables are destroyed after each call, static globals are not

C. Both have program-long lifetime, but static local has function/block scope while static global has file scope

D. Static local variables have file scope; static global variables have block scope

Ans: C. Both have program-long lifetime, but static local has function/block scope while static global has file scope

Explanation: Both share extended lifetime due to static storage duration, but differ in scope: local (function) vs file-wide (global).

Q95 | Statement-based | Static Storage Class — Global Variables

Statement: "Declaring a variable static at global scope changes its lifetime compared to a plain global variable." This is:

A. True — static changes lifetime from program-long to block-long

B. False — both static and plain globals already have program-long lifetime; static only changes linkage/scope, not lifetime

C. True — static globals exist only during compilation

D. True — only static globals persist across program runs

Ans: B. False — both static and plain globals already have program-long lifetime; static only changes linkage/scope, not lifetime

Explanation: Plain global variables already have program-long lifetime by default; adding static only restricts linkage, not lifetime.

5. External Storage Class (extern) 25 Qs

Q96 | Definition | External Storage Class (extern)

What is the primary purpose of the extern storage class in C?

A. To create a temporary local variable

B. To declare a variable that is stored in a CPU register

C. To restrict a variable's scope to one function

D. To declare a reference to a global variable defined in another file or elsewhere in the program, enabling variable sharing

Ans: D. To declare a reference to a global variable defined in another file or elsewhere in the program, enabling variable sharing

Explanation: extern is used to tell the compiler that the variable is defined elsewhere (often another file) and should simply be referenced here.

Q97 | Declaration | External Storage Class (extern)

Which of the following correctly declares (without defining) an external variable?

A. extern int total;

B. int extern total = 5;

C. static extern int total;

D. register extern int total;

Ans: A. extern int total;

Explanation: 'extern int total;' is a pure declaration — it tells the compiler the variable exists elsewhere, without allocating new storage.

Q98 | Concept | External Storage Class (extern)

When you write 'extern int x;' in a file without an initializer, does this allocate new memory for x?

A. Yes, but only 1 byte

B. Yes, memory is always allocated

C. No, it deletes the variable

D. No, it only declares that x is defined somewhere else; no new memory is created

Ans: D. No, it only declares that x is defined somewhere else; no new memory is created

Explanation: A pure extern declaration is just a reference/announcement; the actual memory is allocated wherever the variable is defined.

Q99 | Purpose | External Storage Class (extern)

The extern keyword is primarily useful in programs that:

A. Span multiple source files and need to share global variables between them

B. Only use static local variables

C. Do not use any global variables at all

D. Consist of a single source file only

Ans: A. Span multiple source files and need to share global variables between them

Explanation: extern's main use-case is enabling multiple .c files in a project to access the same global variable.

Q100 | Characteristics | External Storage Class (extern)

If a variable is declared 'extern int count;' in file2.c, and defined as 'int count = 10;' in file1.c, what happens?

A. Each file gets its own independent 'count'

B. file2.c can access and modify the same 'count' that is defined in file1.c

C. This is invalid syntax and won't compile

D. A linking error occurs because count is defined twice

Ans: B. file2.c can access and modify the same 'count' that is defined in file1.c

Explanation: extern correctly links file2.c's reference to the single definition of count present in file1.c, sharing the same variable.

Q101 | Scope | External Storage Class (extern)

The scope of an extern-declared variable, once properly linked to its definition, is:

A. Restricted to a single function only

B. Effectively as wide as the files that declare it as extern, since it refers to a single shared global

C. Restricted to a single block

D. Restricted to the CPU register

Ans: B. Effectively as wide as the files that declare it as extern, since it refers to a single shared global

Explanation: Once linked, the variable is accessible in every file that has an extern declaration referencing the shared global definition.

Q102 | Lifetime | External Storage Class (extern)

What is the lifetime of a variable accessed through extern?

A. Only during the function using it

B. Determined freshly in every file

C. Same as the underlying global variable it refers to — the entire program duration

D. Only until the file finishes execution

Ans: C. Same as the underlying global variable it refers to — the entire program duration

Explanation: extern is just a reference to an already-existing global variable, so its lifetime matches that global's program-long lifetime.

Q103 | Variable sharing | External Storage Class (extern)

Which storage class specifically enables sharing of a single global variable across multiple .c files in a project?

A. extern

B. register

C. auto

D. static

Ans: A. extern

Explanation: extern is designed exactly for this purpose — referencing and sharing global variables across translation units.

Q104 | File accessibility | External Storage Class (extern)

Suppose a global variable is declared as 'static int val;' in file1.c. Can file2.c access it using 'extern int val;'?

A. No, static gives internal linkage, so extern in another file cannot access it

B. Yes, extern overrides static and grants access

C. Yes, but only if val is initialized to 0

D. Yes, if both files are in the same folder

Ans: A. No, static gives internal linkage, so extern in another file cannot access it

Explanation: static's internal linkage strictly prevents access from other files, and this cannot be bypassed by declaring extern elsewhere.

Q105 | Statement-based | External Storage Class (extern)

Statement: "extern can be used to declare a variable multiple times across different files, but it must be defined (without extern) in exactly one file." This is:

A. True only for pointer variables

B. True only for arrays

C. True

D. False

Ans: C. True

Explanation: Multiple extern declarations are allowed, but the C 'one-definition rule' still requires exactly one actual definition of the variable.

Q106 | Conceptual | External Storage Class (extern)

Which of the following would be considered a 'definition' rather than a mere 'declaration'?

A. extern int x;

B. Both (b) and (c) allocate storage and are definitions

C. int x; (at file scope, no extern, may have an initializer)

D. extern int x = 5; (has an initializer, so it IS a definition too)

Ans: B. Both (b) and (c) allocate storage and are definitions

Explanation: A declaration with extern and no initializer is a pure declaration; but 'int x;' or any declaration with an initializer allocates storage and is a definition — including 'extern int x = 5;'.

Q107 | Application | External Storage Class (extern)

extern variables are commonly used in large software projects to:

A. Force variables into CPU registers

B. Share configuration or state variables like error codes or global counters between multiple .c files

C. Avoid the need for header files entirely

D. Make all local variables visible in every file

Ans: B. Share configuration or state variables like error codes or global counters between multiple .c files

Explanation: Sharing state such as global error flags/counters across modules is a very common real-world use of extern.

Q108 | Assertion-Reason | External Storage Class (extern)

Assertion (A): Declaring the same global variable as extern in many files does not cause a 'multiple definition' linker error.
Reason (R): extern declarations (without initializers) do not allocate storage; they only reference an existing definition.

A. Both A and R true, R correctly explains A
B. A false, R true
C. A true, R false
D. Both A and R true, but R does not explain A

Ans: A. Both A and R true, R correctly explains A

Explanation: Because extern declarations don't create new storage, having many of them across files causes no conflict, which explains assertion A.

Q109 | Tricky | External Storage Class (extern)

What error occurs if a variable is defined (with storage, e.g., 'int total = 0;') in two different files without using extern or static, and both files are linked together?

A. The second definition is silently ignored
B. A 'multiple definition' linker error, since each file allocates separate storage for the same external name
C. No error, both are simply merged
D. A compile-time syntax error

Ans: B. A 'multiple definition' linker error, since each file allocates separate storage for the same external name

Explanation: Two full definitions of the same externally-linked global cause a linker conflict due to duplicate storage allocation for the same symbol.

Q110 | Comparison | External Storage Class (extern)

How does extern differ from static at global scope?

A. extern enables sharing across files (external linkage); static restricts visibility to one file (internal linkage)
B. extern is used only for local variables
C. They are functionally identical
D. extern restricts visibility to one file; static allows sharing across files

Ans: A. extern enables sharing across files (external linkage); static restricts visibility to one file (internal linkage)

Explanation: extern and static are essentially opposites in terms of linkage: extern allows cross-file access, static blocks it.

Q111 | Fill-in-the-blank | External Storage Class (extern)

To use a global variable defined in another source file, a program must include a(n) _____ declaration of that variable.

A. extern
B. register
C. auto
D. static

Ans: A. extern

Explanation: The extern keyword is used exactly for referencing variables defined elsewhere, typically in another file.

Q112 | Scenario-based | External Storage Class (extern)

A project has a global error-counter 'errCount' defined once in main.c. Other files (utils.c, io.c) need to increment it. What should utils.c and io.c include?

A. auto int errCount;

B. register int errCount;

C. static int errCount;

D. extern int errCount;

Ans: D. extern int errCount;

Explanation: Each of the other files should declare 'extern int errCount;' to reference the single shared definition in main.c.

Q113 | Previous-Year Pattern | External Storage Class (extern)

Which of the following is the correct relationship between a variable's declaration and definition when using extern?

A. A declaration and definition are always the same statement

B. extern always allocates its own separate storage

C. A variable can never be declared more than once

D. extern int x; is a declaration (no storage); int x; elsewhere is the definition (allocates storage)

Ans: D. extern int x; is a declaration (no storage); int x; elsewhere is the definition (allocates storage)

Explanation: This correctly captures how extern (declaration only) relates to the actual definition that reserves memory.

Q114 | Difficult | External Storage Class (extern)

If 'extern int x;' appears in a file, but 'x' is never actually defined anywhere in the whole program, what happens?

A. A linker error occurs — 'undefined reference to x'

B. A compile-time syntax error occurs immediately

C. It compiles and runs fine with x defaulting to 0

D. x is automatically treated as a local variable

Ans: A. A linker error occurs — 'undefined reference to x'

Explanation: Since extern is only a promise that a definition exists elsewhere, the linker fails if no matching definition is ever found.

Q115 | Conceptual | External Storage Class (extern)

Function declarations (prototypes) in C header files are implicitly treated with which storage-class-like linkage, similar in spirit to extern variables?

A. extern, by default — functions are externally linked unless declared static

B. register, by default

C. auto, by default

D. static, by default

Ans: A. extern, by default — functions are externally linked unless declared static

Explanation: Just like variables, functions have external linkage by default, mirroring how extern variables work across files.

Q116 | Application | External Storage Class (extern)

Global constants meant to be used across many modules of a large C project are typically declared in a header file using:

A. register

B. auto

C. No storage class is ever used for shared constants

D. extern (declaration in header, single definition in one .c file)

Ans: D. extern (declaration in header, single definition in one .c file)

Explanation: This extern-in-header, define-in-one-file pattern is the standard idiom for sharing global variables safely across a multi-file project.

Q117 | Tricky | External Storage Class (extern)

Can a variable be declared as both 'static' and 'extern' simultaneously for the same declaration?

A. Yes, but only for arrays

B. No, static and extern represent conflicting linkage specifications and cannot both apply to the same declaration

C. Yes, and it behaves like a normal extern variable

D. Yes, and it forces the variable into a CPU register

Ans: B. No, static and extern represent conflicting linkage specifications and cannot both apply to the same declaration

Explanation: static (internal linkage) and extern (external linkage) are mutually exclusive linkage specifications for the same declaration.

Q118 | Comparison | External Storage Class (extern)

Among auto, register, static, and extern, which storage class is the only one that can span multiple source files for a single shared variable?

A. static

B. extern

C. auto

D. register

Ans: B. extern

Explanation: extern uniquely allows a single variable's definition to be referenced and shared from multiple different files.

Q119 | Statement-based | External Storage Class (extern)

Statement: "An extern declaration inside a function body can be used to access a global variable defined elsewhere, even if that variable is not visible at file scope in the current file." This is:

A. True only in main()

B. False, extern declarations are only allowed at file scope

C. True

D. True only for pointers

Ans: C. True

Explanation: extern declarations are also permitted inside function bodies (block scope) as a way to access an external global variable locally.

Q120 | Difficult | External Storage Class (extern)

In the statement 'extern int x = 10;' written at file scope, what actually happens?

A. It is treated as a definition of x with external linkage and an initial value of 10, not merely a declaration

B. The initializer 10 is ignored because extern is present

C. It behaves exactly like 'static int x = 10;'

D. This is invalid syntax and never compiles

Ans: A. It is treated as a definition of x with external linkage and an initial value of 10, not merely a declaration

Explanation: Providing an initializer along with extern makes it a definition (with external linkage) rather than a pure declaration.

6. Scope Rules & Storage Class Comparison

28 Qs

Q121 | Definition | Scope Rules & Storage Class Comparison

In C, 'scope' of a variable refers to:

A. Whether it is initialized or not

B. How long the variable exists in memory

C. The amount of memory it occupies

D. The region of the program in which the variable is accessible/visible

Ans: D. The region of the program in which the variable is accessible/visible

Explanation: Scope specifically defines the portion of source code where a variable's name can be legally referenced.

Q122 | Definition | Scope Rules & Storage Class Comparison

In C, 'lifetime' (storage duration) of a variable refers to:

A. Its data type

B. The region of code where it is visible

C. The duration for which the variable actually exists in memory during execution

D. The number of times it is used in a program

Ans: C. The duration for which the variable actually exists in memory during execution

Explanation: Lifetime concerns how long memory for the variable remains allocated during program execution, distinct from scope.

Q123 | Conceptual | Scope Rules & Storage Class Comparison

Which of the following pairs represents the concepts of 'accessibility (scope)' versus 'existence duration (lifetime)'?

A. Scope versus lifetime

B. Declaration versus definition, only

C. Data type versus storage class

D. Global versus local, only

Ans: A. Scope versus lifetime

Explanation: Scope and lifetime are the two independent but related concepts governing how variables can be used.

Q124 | Local Scope | Scope Rules & Storage Class Comparison

A variable with local scope is accessible:

A. In every file of the project

B. Throughout the entire program

C. Only inside main()

D. Only within the block/function in which it is declared

Ans: D. Only within the block/function in which it is declared

Explanation: Local scope confines a variable's visibility to its enclosing block or function.

Q125 | Global Scope | Scope Rules & Storage Class Comparison

A variable with global scope, declared at file level without static, is accessible:

A. Only inside the function where declared

B. Nowhere outside main()

C. Throughout the file, and in other files too if declared there with extern

D. Only inside loops

Ans: C. Throughout the file, and in other files too if declared there with extern

Explanation: Non-static globals have external linkage, making them potentially accessible file-wide and across files via extern.

Q126 | Block Scope | Scope Rules & Storage Class Comparison

Block scope means a variable declared inside { } (e.g., inside an if or for statement) is:

A. Visible in the entire file

B. Visible throughout the entire function

C. Visible in all functions

D. Visible only within that specific block and any nested blocks inside it

Ans: D. Visible only within that specific block and any nested blocks inside it

Explanation: A block-scoped variable's visibility ends at the closing brace of the block in which it is declared.

Q127 | File Scope | Scope Rules & Storage Class Comparison

A variable with file scope is declared:

A. Inside a for-loop only

B. Only using the register keyword

C. Outside all functions, at the top level of a source file

D. Inside a function

Ans: C. Outside all functions, at the top level of a source file

Explanation: File scope applies to identifiers declared outside any function body, at the outermost level of a translation unit.

Q128 | Visibility Rules | Scope Rules & Storage Class Comparison

If a local variable has the same name as a global variable, which one is used inside the function where the local variable is declared?

A. Neither can be used; this is illegal

B. Both are used simultaneously, causing an error

C. The global variable always takes priority

D. The local variable, which 'shadows'/hides the global variable within that block

Ans: D. The local variable, which 'shadows'/hides the global variable within that block

Explanation: Variable name shadowing rules in C give priority to the more locally-scoped declaration within its own block.

Q129 | Accessibility Rules | Scope Rules & Storage Class Comparison

Which storage class specifier used at global scope makes a variable inaccessible from other source files?

A. auto

B. static

C. register

D. extern

Ans: B. static

Explanation: static gives global variables internal linkage, restricting accessibility strictly to the declaring file.

Q130 | Lifetime versus Scope | Scope Rules & Storage Class Comparison

Which of the following correctly illustrates the difference between lifetime and scope using the static local variable?

A. A static local variable has file scope and block lifetime

B. A static local variable has program-long scope and block lifetime

C. A static local variable has block/function scope but program-long lifetime

D. A static local variable has no scope and no lifetime

Ans: C. A static local variable has block/function scope but program-long lifetime

Explanation: This is the textbook example: extended (program-long) lifetime combined with restricted (function-only) scope.

Q131 | Comparison | Scope Rules & Storage Class Comparison

Which storage classes provide local scope AND local (function-call) lifetime together?

A. static (global) only

B. auto and register

C. static (local) only

D. extern only

Ans: B. auto and register

Explanation: Both auto and register variables share local scope and lifetime tied to the function call, unlike static or extern.

Q132 | Comparison | Scope Rules & Storage Class Comparison

Which storage class provides file/global scope AND program-long lifetime, but restricted to a single file?

A. register

B. extern

C. auto

D. static (global)

Ans: D. static (global)

Explanation: Static global variables combine program-long lifetime with scope confined to a single file (internal linkage).

Q133 | Comparison | Scope Rules & Storage Class Comparison

Which storage class provides potentially multi-file scope AND program-long lifetime?

A. register

B. auto

C. static

D. extern (referring to an externally-linked global)

Ans: D. extern (referring to an externally-linked global)

Explanation: extern-linked global variables can be accessed from multiple files and persist for the whole program run.

Q134 | Match-the-Following | Scope Rules & Storage Class Comparison

Match: (1) auto (2) register (3) static local (4) extern — with: (i) Multi-file sharing (ii) CPU-register hint, no address-of (iii) Block scope, block lifetime (iv) Block scope, program lifetime. Which is the correct match?

A. 1-ii, 2-iii, 3-i, 4-iv

B. 1-iii, 2-ii, 3-iv, 4-i

C. 1-iv, 2-i, 3-ii, 4-iii

D. 1-i, 2-ii, 3-iii, 4-iv

Ans: B. 1-iii, 2-ii, 3-iv, 4-i

Explanation: auto→block scope+lifetime; register→CPU hint with no address-of; static local→block scope+program lifetime; extern→multi-file sharing.

Q135 | Match-the-Following | Scope Rules & Storage Class Comparison

Match: (1) Local Scope (2) Global Scope (3) Block Scope (4) File Scope — with: (i) Declared outside all functions (ii) Confined to { } braces (iii) Confined to a function (iv) Program-wide (typically) via external linkage. Correct match?

A. 1-ii, 2-i, 3-iv, 4-iii
B. 1-iii, 2-iv, 3-ii, 4-i
C. 1-i, 2-ii, 3-iii, 4-iv
D. 1-iv, 2-iii, 3-i, 4-ii

Ans: B. 1-iii, 2-iv, 3-ii, 4-i

Explanation: Local scope→confined to a function; Global scope→program-wide via linkage; Block scope→within braces; File scope→declared outside all functions.

Q136 | Statement-based | Scope Rules & Storage Class Comparison

Statement: "All variables with file scope automatically have program-long lifetime." This statement is:

A. True
B. True only if extern is used
C. True only for arrays
D. False

Ans: A. True

Explanation: Any variable declared at file scope (static or not) has static storage duration, meaning it exists for the entire program run.

Q137 | Statement-based | Scope Rules & Storage Class Comparison

Statement: "A variable's scope can be block-level even though its lifetime is program-long." This statement is:

A. True — this describes static local variables exactly
B. False, scope and lifetime must always match
C. True only for extern variables
D. True only for register variables

Ans: A. True — this describes static local variables exactly

Explanation: Static local variables are the precise real-world example proving that scope and lifetime can differ within one variable.

Q138 | Assertion-Reason | Scope Rules & Storage Class Comparison

Assertion (A): Scope and lifetime are independent properties of a variable.
Reason (R): A static local variable has restricted (block) scope but extended (program-long) lifetime, showing the two need not coincide.

A. Both A and R true, R correctly explains A
B. A true, R false
C. A false, R true
D. Both A and R true, but R does not explain A

Ans: A. Both A and R true, R correctly explains A

Explanation: The static local variable example is a direct, correct illustration that scope and lifetime are separate concepts.

Q139 | Comparison | Scope Rules & Storage Class Comparison

Complete the comparison table entry: Storage Class = auto | Scope = Local | Lifetime = ? | Default Value = ?

A. File-only ; NULL
B. Multi-file ; 0
C. Block/function ; Garbage (indeterminate)
D. Program-long ; 0

Ans: C. Block/function ; Garbage (indeterminate)

Explanation: auto variables exist only for the block's execution and are not automatically zero-initialized (garbage default).

Q140 | Comparison | Scope Rules & Storage Class Comparison

Complete the comparison table entry: Storage Class = static (local) | Scope = ? | Lifetime = ? | Default Value = ?

A. Block/function ; Program-long ; 0
B. Global ; Block-only ; garbage
C. Multi-file ; Block-only ; 0
D. File ; Program-long ; garbage

Ans: A. Block/function ; Program-long ; 0

Explanation: Static local variables have restricted block scope but program-long lifetime and are zero-initialized by default.

Q141 | Comparison | Scope Rules & Storage Class Comparison

Complete the comparison table entry: Storage Class = extern | Scope = ? | Lifetime = ?

A. CPU register ; Program-long
B. Block-only ; Block-only
C. File-only ; Block-only
D. Program-wide (multi-file, via linkage) ; Program-long

Ans: D. Program-wide (multi-file, via linkage) ; Program-long

Explanation: extern-referenced global variables can be scoped across multiple files and persist for the program's entire run.

Q142 | Tricky | Scope Rules & Storage Class Comparison

Which of the following variables would have BOTH the widest possible scope AND the longest lifetime simultaneously?

A. A non-static global variable accessed via extern from multiple files
B. An auto variable inside a nested loop
C. A static local variable
D. A register variable inside a function

Ans: A. A non-static global variable accessed via extern from multiple files

Explanation: A plain (non-static) global variable shared through extern declarations achieves both multi-file scope and program-long lifetime.

Q143 | Difficult | Scope Rules & Storage Class Comparison

Rank the following storage classes from MOST restricted scope to LEAST restricted scope: auto/register (block), static local (block), static global (file), extern-linked global (multi-file).

A. auto/register = static local (block) < static global (file) < extern-linked global (multi-file/program)
B. extern < static global < static local = auto/register
C. static global < auto/register < extern < static local
D. All have identical scope

Ans: A. auto/register = static local (block) < static global (file) < extern-linked global (multi-file/program)

Explanation: Block-scoped variables (auto, register, static local) are most restricted; static global is file-wide; extern-linked globals can span the whole program.

Q144 | Scenario-based | Scope Rules & Storage Class Comparison

A function-local variable needs to (a) be visible only inside that function, and (b) retain its value across repeated calls. Which single storage class satisfies BOTH requirements?

A. auto
B. static
C. register
D. extern

Ans: B. static

Explanation: static local variables uniquely combine restricted local scope with persistent, program-long lifetime.

Q145 | Previous-Year Pattern | Scope Rules & Storage Class Comparison

Which of the following correctly ranks storage classes by their typical use for cross-file variable sharing (from 'not shareable' to 'fully shareable')?

A. static global is the most shareable of all
B. auto/register and static local (not shareable) < static global (not shareable outside file) < plain global with extern (shareable across files)
C. register variables are the most shareable across files
D. extern < static (global) < auto/register

Ans: B. auto/register and static local (not shareable) < static global (not shareable outside file) < plain global with extern (shareable across files)

Explanation: Block-scoped classes are never shareable across files; static globals stay confined to one file; extern enables true cross-file sharing.

Q146 | Conceptual | Scope Rules & Storage Class Comparison

Which two storage classes share identical scope and lifetime rules, differing only in a compiler storage hint?

A. extern and register
B. auto and register
C. static and extern
D. static local and static global

Ans: B. auto and register

Explanation: auto and register are functionally identical in scope/lifetime; register merely adds a (non-binding) CPU-register storage suggestion.

Q147 | Difficult | Scope Rules & Storage Class Comparison

A programmer removes the 'static' keyword from a previously static local variable in a function that is called repeatedly in a loop, expecting a running total. What is the most likely consequence?

A. The variable resets to its initial value on every call (loses its 'running total' behavior), since it is now auto by default
B. No change in behavior at all
C. The program fails to compile
D. The variable becomes accessible from other files

Ans: A. The variable resets to its initial value on every call (loses its 'running total' behavior), since it is now auto by default

Explanation: Without static, the variable reverts to auto behavior, losing its persisted value and no longer functioning as a running total.

Q148 | Tricky | Scope Rules & Storage Class Comparison

Why is it generally considered good practice to minimize the use of plain (non-static) global variables in large C programs, even though extern allows sharing them across files?

A. Because widespread global scope increases the risk of unintended modification and naming conflicts across a large codebase

B. Because global variables cannot store integer values

C. Because extern variables are always slower than local variables

D. Because global variables are illegal in ISO C

Ans: A. Because widespread global scope increases the risk of unintended modification and naming conflicts across a large codebase

Explanation: Wide, uncontrolled scope raises maintenance risk (accidental writes, name collisions), which is why controlled sharing/encapsulation is preferred.